

Short technical report

Official template: general information, feature checklist, architecture, empirical evidence and challenges.

1. GENERAL INFORMATION & DELIVERABLE LINKS

Course: Cross-Platform Mobile App Development (VKU)

Mini-Project Title: Mini-Project 3 - OCR Expense Tracker & Transfer Parser

Student: Dương Bảo Đạt | **Student ID:** 23IT046

Team: Individual | **Role:** Full-stack mobile developer | **Contribution:** 100%

Submission date: 09/10/2026 | **Version:** 1.1.0

GitHub: <https://github.com/duongdatdev/transfer-lens>

APK + video: [GitHub Release v1.1.0](#) - download all deliverables

PDF: [transfer-lens-report.pdf](#) (this report)

Adapted scenario: Instead of paper receipts, the user imports a completed bank-transfer image. The parser extracts amount, date, sender/recipient and description. A keyword rule suggests a category; unknown descriptions require manual selection. This scope change follows the student request and is documented for instructor review.

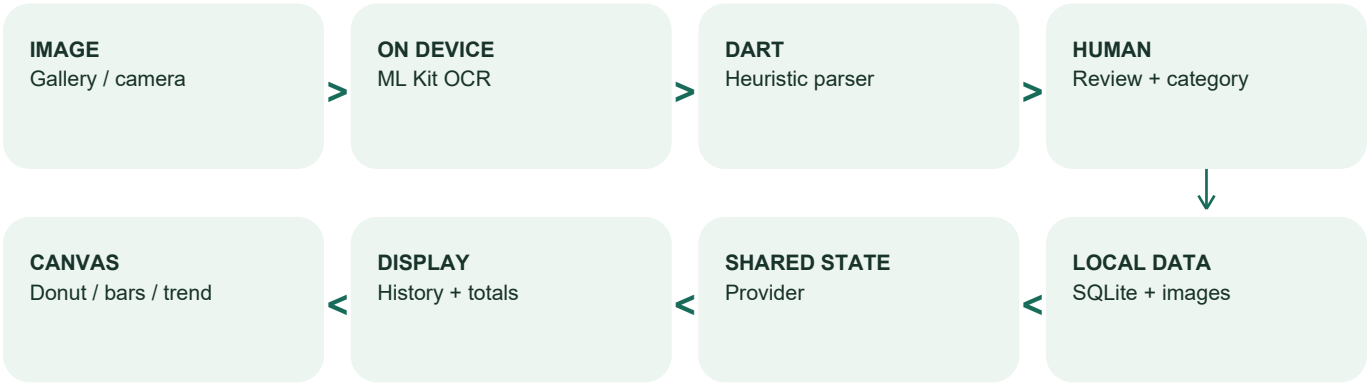
2. FEATURE IMPLEMENTATION CHECKLIST

Required feature	Status	Implementation / acceptance
Image input + crop	Implemented	Gallery screenshot import, native crop/rotate; optional camera with flash, focus and frame.
On-device OCR	Validated	Bundled ML Kit Latin model; native Android OCR exercised on fictional image assets.
Regex heuristics	Validated	VND, calendar date, explicit parties, description/reference; ambiguity and failure warnings.
Category + review	Validated	Five mapped categories plus Other; editable fields, confirmation and duplicate warning.
Provider + SQLite CRUD	Validated	Local records survive database reopen; full-size images and resized thumbnails cached privately.
Canvas visualizations	Validated	Animated donut, weekly bars and daily income/expense trend; tap inspection and replay.
Material 3 + accessibility	Validated	Light/dark/system, responsive layout, large-text and reduced-motion widget checks.
Release + report	Published	Signed APK, 2-3 minute native walkthrough, four-page report with regex table.

Architecture & validation

A modular Flutter application with local data ownership and transparent parsing rules.

3. TECHNICAL ARCHITECTURE & PROJECT STRUCTURE



Approved review → Provider ExpenseStore → SQLite repository + private image storage → history + CustomPainter charts.

Direction defaults to expense and remains editable. The donut/bars exclude income; the daily trend separates income and expenses. It shows recorded cash flow, not the bank balance.

Directory	Responsibility
lib/domain	Transaction model, VND/date/status rules, description keywords, calendar-day cash-flow aggregation.
lib/data + state	SQLite repository and constraints; Provider transactions, duplicate warning and persistent theme.
lib/services	Native OCR lifetime; private originals; isolate-based thumbnail resizing.
lib/ui + widgets	Import/review/history; donut, bars and trend painters; localized animation and reduced motion.

Failure handling: Uncertain fields remain editable. Review and image caching precede database writes; failed writes clean new images. Deletion removes the row before files. Camera/OCR resources are disposed; Android backup is disabled.

Measured verification

Toolchain: Flutter 3.47.5, Dart 3.13.4, SDK 36, Java 21; Android API 24+.

Checks: Analyzer; 41 unit/widget tests; native Android walkthrough. Trend tests cover leap years, month boundaries, totals, interrupted transitions, replay, 200% text and reduced motion.

OCR: v1.0.0 signed release: 2,595 ms offline; initial emulator cold run: 4,488 ms. Latest demo measurements follow; these are not physical-device latency claims.

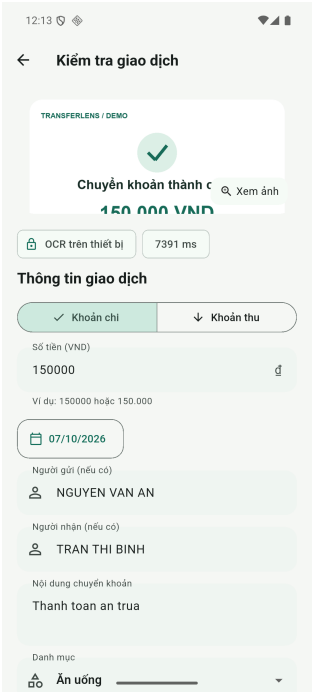
- DEMO-OTHER-003: 2632 ms
- DEMO-FOOD-001: 7391 ms
- DEMO-STUDY-002: 2996 ms
- DEMO-INCOME-004: 3675 ms

Empirical evidence

Actual screenshots from the Android emulator; the banking confirmations contain fictional demo data.

4. EMPIRICAL EVIDENCE & SCREENSHOTS

A. Native OCR review



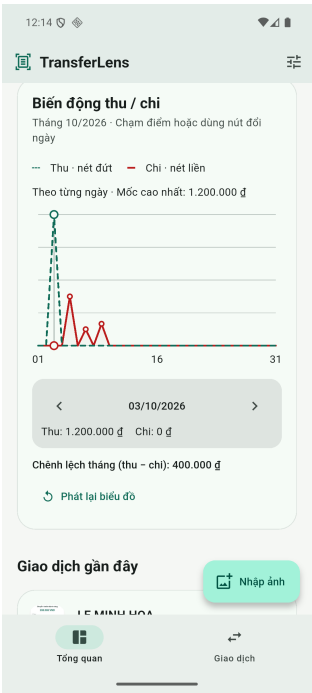
The imported image, measured OCR duration and editable amount appear before saving.

B. Manual classification



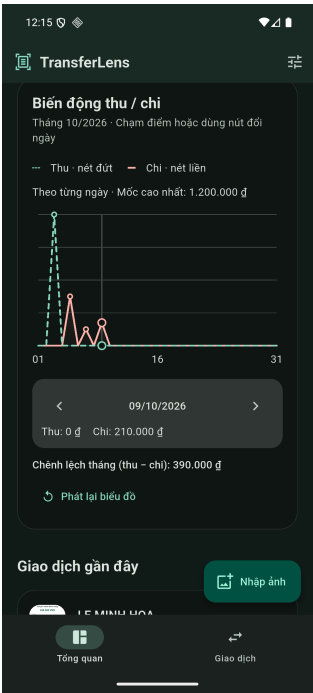
A generic transfer description has no confident keyword match, so a category must be selected.

C. Animated daily trend



Real saved records form income (dashed) and expense (solid) series. Select a date or replay the reveal.

D. Dark mode + updates



Edited amounts change the trend and monthly net. Theme changes retain the same financial data.

Challenges & resolutions

Design decisions, validation limits and the practical submission handoff.

5. TECHNICAL CHALLENGES & RESOLUTIONS

Financial ambiguity: Explicit amount labels outrank currency-only values; balance, fee, account and reference fields are excluded. Conflicting amounts or invalid dates require review. Only explicitly labeled parties are extracted.

Animation and builds: Daily values interpolate from the current frame during rapid updates. Calendar indexes handle leap years. Flexible legends support 200% text. R8 preserves native ML Kit registrar constructors; Windows Kotlin incremental caching is disabled across drives.

OCR regex and heuristic table

Field	Rule / regex after normalization	Fallback
VND amount	<code>\d{1,3}(?:[.]\d{3})+ or plain digits; prioritize so tien / amount labels.</code>	Reject malformed grouping; manual amount.
Date	<code>(\d{1,2})[/.-](\d{1,2})[/.-](\d{4}); validate calendar + 2000-2100.</code>	Invalid/conflicting dates: date picker.
Parties / content	Label prefixes: <code>nguoì gui, nguoì nhan, noi dung</code> ; read adjacent lines.	Missing fields remain editable.
Category	Accent-free whole phrases: <code>an trua, hoc phi, taxi, mua sam, xem phim.</code>	No unique match: manual category.
Completion	<code>thanh cong successful completed; failed/pending override success.</code>	Always require human confirmation.

Acceptance scope and remaining checks

Physical-device follow-up: Camera flash/focus, gallery/crop and real-bank image accuracy still require a target phone. The sub-100ms target has not been demonstrated. iOS requires macOS/Xcode validation.

Privacy: No cloud OCR or bank API. SQLite, images and raw text stay local; backups are disabled, with no additional file encryption. Records are user-confirmed and do not prove payment.

Build and demonstrate

Setup: `flutter pub get; flutter run -d <android-device>`.

Quality: `flutter analyze; flutter test`. Native tests: `integration_test/app_test.dart` and `demo_test.dart`.

Release: Set private android/key.properties; `flutter build apk --release`.

Video: Actual native screens, real ML Kit and fictional data; reproducible integration-test recording.

References

Google ML Kit: [bundled Android text-recognition model](#)

Flutter ML Kit plugin: [mobile platforms and model setup](#)

Flutter CustomPainter: [canvas rendering API](#)

Original report structure: [supplied Mini-Project-3-Report-Template.md](#)

Course slides: [Week 7 Part 1 \(rubric p.42\)](#); [Week 8 Part 2 \(canvas pp.30-34, checklist p.43\)](#)